

sec-npm: The Zero-Trust Sentinel

A Deep-Dive into Advanced Supply-Chain Defense Architecture

Project Name: sec-npm

Core Concept: Passive Observation vs. Active Deception

Date: May 6, 2026

Classification: Technical Whitepaper

Executive Summary

Traditional security tools rely on reactive databases of *known* vulnerabilities. **sec-npm** introduces a proactive, heuristic-driven engine that identifies malicious intent in real-time. By combining AST de-obfuscation with a high-fidelity shadow sandbox and "Honey-Trap" deception, sec-npm provides a definitive shield against Zero-Day supply-chain hijacks.

Contents

1	Introduction: The Modern Threat Landscape	2
2	The Four Pillars of Defense	2
2.1	1. The Similarity Guard (Typosquatting Detection)	2
2.2	2. Deep Static Intelligence (AST & De-obfuscation)	2
2.3	3. Time-Travel Differential Analysis	2
2.4	4. Shadow Execution (Active Deception)	3
3	Lifecycle Hook Coverage: The Danger Zones	3
4	Operational Philosophy: Heuristics over Blacklists	3
5	Case Study: Auditing a Reputable Package (Axios)	4
6	Conclusion	4

1 Introduction: The Modern Threat Landscape

The JavaScript ecosystem is powered by millions of dependencies, creating a massive, interconnected trust network. However, this network is increasingly fragile. The incidents involving `axios` metadata hijacks and `ua-parser-js` malware proved that even the most reputable packages can be weaponized in minutes.

Hackers no longer attack the front door; they attack the **Supply Chain**. They hijack maintainer accounts, publish malicious updates, and vanish before the community notices. **sec-npm** was designed to be the "Security Sentry" that interrogates every package before it is allowed to enter the developer's environment.

2 The Four Pillars of Defense

2.1 1. The Similarity Guard (Typosquatting Detection)

The first line of defense is the **Levenshtein Similarity Engine**. It analyzes the package name DNA to prevent "Brand Jacking."

- ✓ **Heuristic:** If a package name is ≤ 2 characters away from a top-tier package (e.g., `axois` vs `axios`), it is flagged as a Critical Risk.
- ✓ **Efficacy:** This stops the most common form of "Spray-and-Pray" attacks where users make simple typing errors.

2.2 2. Deep Static Intelligence (AST & De-obfuscation)

Sophisticated malware hides its code using obfuscation. Traditional text-based scanners fail here. **sec-npm** parses the code into an **Abstract Syntax Tree (AST)**.

- **Constant Folding:** The tool "solves" string math (e.g., `'ch' + 'ild_p' + 'rocess'`) to reveal the true underlying API calls.
- **Entropy Analysis:** We measure the randomness of strings. High-entropy strings often hide encrypted payloads or base64 malware.
- **Binary Detection:** The scanner flags the use of `Buffer.from()` and Hex-encoded strings, which are often used to construct malicious executables on the fly.

2.3 3. Time-Travel Differential Analysis

This is the "Historical Memory" of the tool. It compares the current version of a package with its predecessor.

- **The Hijack Fingerprint:** A sudden addition of installation scripts (`postinstall`, `preinstall`) in a package that was historically "clean" is the most accurate indicator of a compromised account.
- **Verification:** By checking the "Delta" of the package footprint, we catch hijacks before the malicious script even runs.

2.4 4. Shadow Execution (Active Deception)

The "Nuclear Option" of sec-npm. Instead of isolation, we use ****Deception****.

- **Honey-Traps:** The sandbox is populated with fake credentials, decoy SSH keys, and mock environment variables.
- **Side-Effect Trapping:** Using ES6 Proxies, we intercept every system call. If a script tries to read a Honey-Trap file or phone home to an external IP, it is caught instantly.
- **Simulated Reality:** The malware thinks it is succeeding, but it is actually performing in a "Digital Theater" where it has no power.

3 Lifecycle Hook Coverage: The Danger Zones

Malware hides in the "Lifecycle Hooks" that npm runs automatically. **sec-npm** monitors the entire spectrum:

- **Installation:** `preinstall`, `install`, `postinstall`.
- **Build/Compiling:** `prepare`, `prepublish`, `prepack`.
- **Destruction (The Parting Gift):** `uninstall`. Malware often tries to wipe the disk when the user discovers the infection and tries to delete the package. We catch this intent in the sandbox before the deletion happens.

4 Operational Philosophy: Heuristics over Blacklists

A major goal of **sec-npm** is to avoid hindering legitimate developers. We use a ****Scoring System**** rather than a binary block. However, certain signals are so high-confidence that they trigger an **Instant Block (Score: 100)**:

- **Honey-Trap Violation:** Any attempt to access decoy credential files in the sandbox.
- **Critical Typosquatting:** Direct name-jacking of top-tier modules (e.g., `axois` vs `axios`).
- **Intelligent Scoring:** For all other cases, we only block when multiple signals overlap (e.g., *New Script* + *Obfuscation* + *Network Access*).
- **Human in the Loop:** The user receives a clear "Green/Yellow/Red" report, allowing them to make the final call with full technical visibility.

5 Case Study: Auditing a Reputable Package (Axios)

To demonstrate the tool's precision, we conducted an audit of **axios@1.16.0**, a widely trusted networking library. The results provide a perfect example of the "Heuristic Intelligence" model.

Audit Result: axios@1.16.0

Risk Score: 50 / 100 **Status:** WARNING

- **Detection:** Identified `prepare:` `husky lifecycle hook`.
- **Findings:** 11 AST anomalies detected (`Buffer.from`, `process.env`).
- **Shadow Result:** 0 Dynamic Anomalies.

Security Interpretation: The tool correctly identified that `axios` uses sensitive patterns (accessing environment variables for proxy settings and using `Buffer` for data payloads). While these patterns are common in malware, the **"Shadow Execution"** confirmed that no malicious intent was present (no Honey-Trap triggers, no unauthorized network calls).

The score of 50 serves as a "Cautionary Flag." For a networking library, this is expected. However, if these same patterns were found in a simple UI component or a utility library, the tool's warning would be the critical signal that prevents a major breach.

6 Conclusion

sec-npm represents a shift from "Detection" to "Deception." By creating a Zero-Trust environment where code must prove its innocence through behavioral simulation, we have built a fortress that is faster, safer, and more intelligent than traditional sandboxing.

Don't just install. Secure.